# MAC (Mandatory Access Control) on Red Hat Enterprise Linux 10 with SELinux


1. Lab Title
2. Objective
3. Background
4. Lab Requirements
5. Phase 1: System Preparation and Registration
    - Purpose
    - Obtaining the RHEL 10 Trial
    - Installing the Operating System
    - Registering the System with Red Hat Subscription Manager
        - Explanation
        - Expected result
        - Additional note
6. Phase 2: Apache Installation and Service Customization
    - Purpose
    - Installing Required Packages
        - Package roles
    - Verifying the Default Apache Page
    - Creating a Custom Web Root Directory
        - Purpose
    - Editing the Apache Configuration
        - Change the listening port
        - Change the document root
        - Change the corresponding directory block
        - Interpretation
    - Verifying Apache Configuration Syntax
    - Restarting Apache
7. Phase 3: Adjusting SELinux Policies for Non-Standard Configuration
    - Purpose
    - Checking the SELinux Mode
    - Allowing Apache to Use Port 3131
        - Explanation

Output
Security significance
Verification
Applying File Contexts to the Custom Web Directory
Explanation of the first command
Explanation of the second command
Verification
**Output**
Security Interpretation
8. Phase 4: Firewall Configuration, Persistence, and Verification
Opening the Firewall for Port 3131
Explanation
Verification
Enabling Apache at Boot
Explanation
Verification
Local Verification
Browser-Based Verification
9. Expected Final Outcome
10. Conclusion
11. Appendix: Command Summary

# 1. Lab Title

**MAC (Mandatory Access Control) on Red Hat Enterprise Linux 10 with SELinux**

# 2. Objective

This lab provides a comprehensive, end-to-end guide to deploying a web server on **Red Hat Enterprise Linux 10 (RHEL 10)** and securing it using **SELinux**. The exercise covers the full lifecycle of the system, beginning with operating system preparation and subscription registration, followed by Apache installation, service customization, SELinux policy adjustment, firewall configuration, and final verification.

The main purpose of this lab is not only to demonstrate how to install and run an Apache web server, but also to illustrate how security controls in enterprise Linux

environments affect service deployment when non-default configurations are introduced.

Upon completion of this lab, the system should satisfy the following conditions:

- RHEL 10 is successfully installed on a virtual machine.
- The system is properly registered with Red Hat Subscription Manager.
- Apache HTTP Server is installed and operational.
- The default Apache listening port is changed from `80` to `3131`.
- The default web root directory is changed from `/var/www/html` to `/var/test_www/html`.
- SELinux policies are updated to allow Apache to use the custom port and directory.
- The firewall is configured to permit traffic on the custom port.
- The web service is accessible through local and external testing.

This lab is particularly valuable because it demonstrates that, in enterprise Linux systems, changing an application's configuration alone is often insufficient. The administrator must also ensure that the security framework, especially SELinux, explicitly permits the new operational behavior.

# 3. Background

In a conventional Linux environment, a web server can often be reconfigured simply by modifying its application files and restarting the service. However, **Red Hat Enterprise Linux** includes **SELinux (Security-Enhanced Linux)**, which enforces a form of **mandatory access control**. Under SELinux, services are restricted not only by traditional UNIX file permissions, but also by security policies that define which processes may access which files, directories, ports, and system resources.

In this lab, Apache is intentionally configured to use:

- a **non-standard port**: `3131`
- a **non-standard web root directory**: `/var/test_www/html`

These modifications are expected to trigger SELinux restrictions unless explicit policy changes are made. Therefore, the lab serves as an applied demonstration of how system administrators must coordinate:

- application configuration
- service management
- firewall rules
- SELinux labeling and policy mapping

This reflects real-world enterprise administration, where system hardening and service functionality must coexist.

# 4. Lab Requirements

Before beginning the lab, the following resources should be prepared:

- A virtual machine capable of running **RHEL 10**
  - Start RHEL Trial: https://www.redhat.com/en/technologies/linux-platforms/enterprise-linux/server/trial
  - Install RHEL on VMWare: https://www.youtube.com/watch?v=Pi0Owi0UHgk
- Internet connectivity for registration and package installation
- A valid Red Hat account or trial subscription
- Administrative privileges (`sudo`) or root access

**Recommended virtual machine specifications**

- **RAM:** 2 GB or more
- **CPU:** 2 vCPUs or more
- **Disk space:** 20 GB or more

---

# 5. Phase 1: System Preparation and Registration

## Purpose

The purpose of this phase is to prepare the operating system environment and ensure that the RHEL system is properly registered with Red Hat so that official repositories and software packages can be accessed.

Without registration, package installation through `dnf` or `yum` may fail because the system will not have access to the required repositories.

## Obtaining the RHEL 10 Trial

The first step is to obtain the RHEL 10 installation media. The administrator should visit the official Red Hat trial page and download the installation ISO for Red Hat Enterprise Linux 10.

After downloading the ISO, it should be attached to the virtual machine using a hypervisor such as VMware or VirtualBox.

## Installing the Operating System

Install RHEL 10 on the virtual machine by following the standard installation workflow. During installation, the administrator should configure:

- system language
- installation target disk
- root password
- user account
- hostname
- timezone

After the installation is completed, log in to the system and verify the operating system version with the following command:

```
cat /etc/redhat-release

Red Hat Enterprise Linux release 10.1 (Coughlan
```

```
[lab@localhost conf]$ cat /etc/os-release
NAME="Red Hat Enterprise Linux"
VERSION="10.1 (Coughlan)"
ID="rhel"
ID_LIKE="centos fedora"
VERSION_ID="10.1"
PLATFORM_ID="platform:el10"
PRETTY_NAME="Red Hat Enterprise Linux 10.1 (Coughlan)"
ANSI_COLOR="0;31"
LOGO="fedora-logo-icon"
CPE_NAME="cpe:/o:redhat:enterprise_linux:10.1"
HOME_URL="https://www.redhat.com/"
VENDOR_NAME="Red Hat"
VENDOR_URL="https://www.redhat.com/"
DOCUMENTATION_URL="https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/10"
BUG_REPORT_URL="https://issues.redhat.com/"

REDHAT_BUGZILLA_PRODUCT="Red Hat Enterprise Linux 10"
REDHAT_BUGZILLA_PRODUCT_VERSION=10.1
REDHAT_SUPPORT_PRODUCT="Red Hat Enterprise Linux"
REDHAT_SUPPORT_PRODUCT_VERSION="10.1"
```

The expected result is a line indicating that the system is running Red Hat Enterprise Linux 10.

## Registering the System with Red Hat Subscription Manager

Once installation is complete, the system must be registered with Red Hat in order to access software repositories and receive updates.

Run the following command:

```
sudo subscription-manager register--username <your_username>--password <your_password>--auto-attach
```

### Explanation

- `subscription-manager register` registers the system with Red Hat.
- `-username` specifies the Red Hat account username.
- `-password` specifies the account password.
- `-auto-attach` automatically attaches an available subscription to the system.

### Expected result

If successful, the system will display a confirmation message indicating that registration has completed and a subscription has been attached.

To verify the registration state, run:

```
sudo subscription-manager status
```

A valid status confirms that the system is ready for package installation.

### Additional note

If repository or GPG-related errors occur, the administrator should verify:

- network connectivity
- Red Hat account validity
- correct subscription attachment

It is also useful to refresh package metadata using:

```
sudo dnf clean all
sudo dnf repolist
```

# 6. Phase 2: Apache Installation and Service Customization

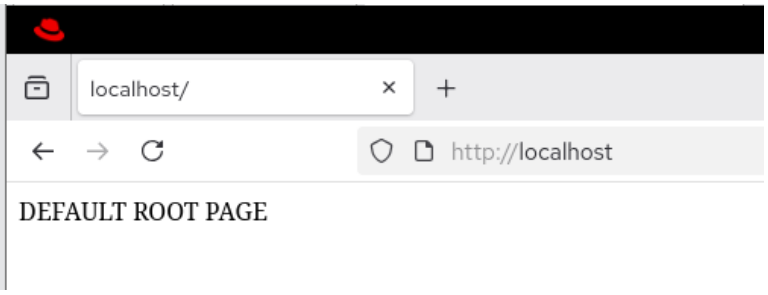
## Purpose

The objective of this phase is to install the Apache web server and intentionally move it away from its default operating configuration. Rather than using port `80` and the directory `/var/www/html`, Apache will be configured to:

- listen on port `3131`
- serve content from `/var/test_www/html`

This configuration provides a controlled scenario for observing how SELinux reacts to non-default service behavior.

**Default root page on port 80**




# Installing Required Packages

Run the following command:

```
sudo yum install httpd policycoreutils-python-utils setrouble
shoot-server-y
```

## Package roles

- `httpd`: installs Apache HTTP Server
- `policycoreutils-python-utils`: provides SELinux management tools such as `semanage`
- `setroubleshoot-server`: helps diagnose SELinux-related access denials

# Verifying the Default Apache Page

Before modifying the service, it is useful to confirm that Apache operates correctly in its default state.

Start the service:

```
sudo systemctlstart httpd
```

Check the service status:

```
sudo systemctl status httpd
```

Then test the default web page using a browser or the following command:

```
curl http://localhost
```

The expected result is the default Apache welcome page or related HTML output. This confirms that the web server is correctly installed before customization begins.

## Creating a Custom Web Root Directory

Create the new directory structure that will replace the default web root:

```
sudo mkdir-p /var/test_www/html
```

Create a sample page for testing:

```
Apache custom web root on RHEL 10"echo" |sudotee /var/test_ww
w/html/index.html
```

### Purpose

This directory will serve as the new content location for Apache. The `index.html` file provides a simple way to confirm whether Apache is actually reading content from the intended custom directory.

## Editing the Apache Configuration

Open the main Apache configuration file:

```
sudo vi /etc/httpd/conf/httpd.conf
```

Apply the following changes.

### Change the listening port

Locate:

```
Listen 80
```

Replace it with:

```
Listen 3131
```

## Change the document root

Locate:

```
DocumentRoot "/var/www/html"
```

Replace it with:

```
DocumentRoot "/var/test_www/html"
```

## Change the corresponding directory block

Locate:

```
<Directory "/var/www/html">
```

Replace it with:

```
<Directory "/var/test_www/html">
```

A valid configuration example is:

```
<Directory "/var/test_www/html">
    AllowOverride None
    Require all granted
</Directory>
```

```
#
# Listen: Allows you to bind Apache to specific IP addresses and/or
# ports, instead of the default. See also the <VirtualHost>
# directive.
#
# Change this to Listen on a specific IP address, but note that if
# httpd.service is enabled to run at boot time, the address may not be
# available when the service starts.  See the httpd.service(8) man
# page for more information.
#
#Listen 12.34.56.78:80
Listen 3131
```

```
#
# DocumentRoot: The directory out of which you will serve your
# documents. By default, all requests are taken from this directory, but
# symbolic links and aliases may be used to point to other locations.
#
DocumentRoot "/var/test_www/html"

#
# Relax access to content within /var/www.
#
<Directory "/var/test_www/html">
    AllowOverride None
    # Allow open access:
    Require all granted
</Directory>

# Further relax access to the default document root:
<Directory "/var/www/html">
    #
    # Possible values for the Options directive are "None", "All",
    # or any combination of:
    #   Indexes Includes FollowSymLinks SymLinksifOwnerMatch ExecCGI MultiViews
    #
    # Note that "MultiViews" must be named *explicitly* --- "Options All"
    # doesn't give it to you.
    #
    # The Options directive is both complicated and important.  Please see
    # http://httpd.apache.org/docs/2.4/mod/core.html#options
    # for more information.

    Options Indexes FollowSymLinks
```

### Interpretation

- `Listen 3131` instructs Apache to listen on the custom port.
- `DocumentRoot` defines the root directory from which web content will be served.
- `<Directory ...>` applies access rules to the selected directory.

## Verifying Apache Configuration Syntax

Before restarting the service, validate the syntax of the Apache configuration:

```
sudo apachectl configtest
```

The expected output is:

```
Syntax OK
```

If syntax errors are reported, they must be corrected before restarting Apache.

## Restarting Apache

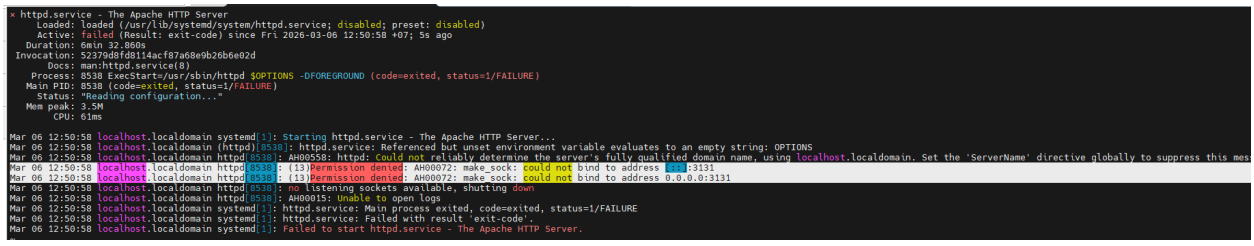
Apply the new configuration by restarting the service:

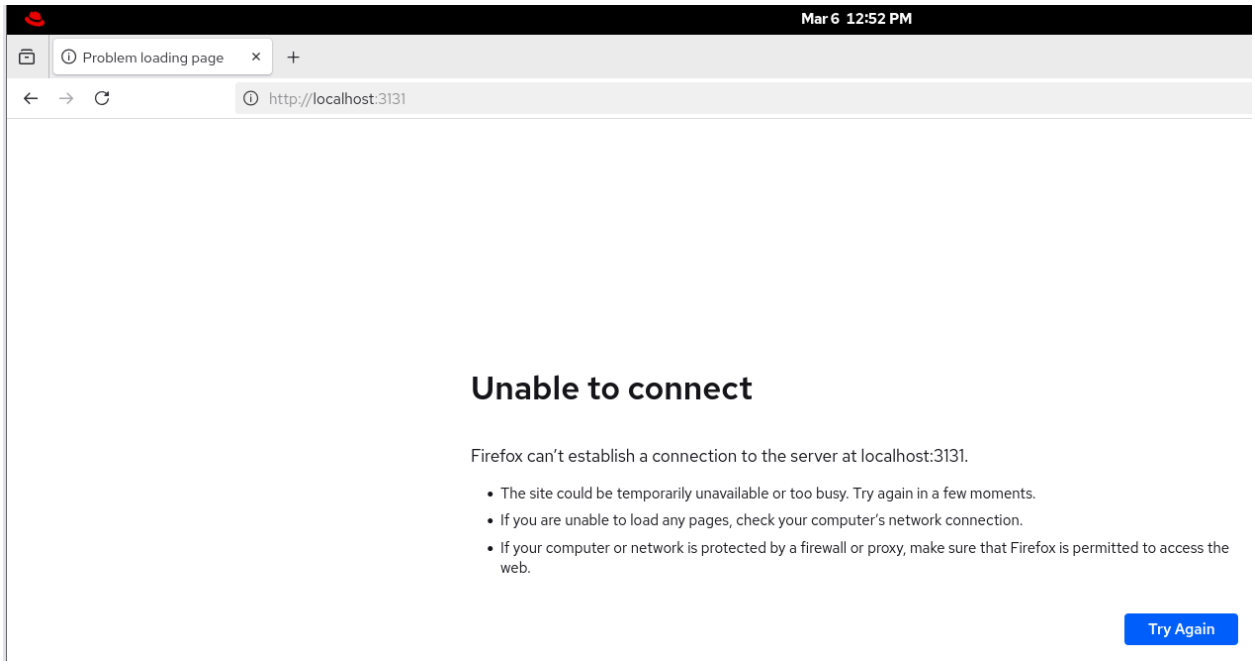
```
sudo systemctl restart httpd
```

Then check the status:

```
sudo systemctl status httpd
```

At this point, if SELinux has not yet been configured, Apache may fail to start correctly or may start but remain inaccessible. This behavior is expected and will be addressed in the next phase.




# 7. Phase 3: Adjusting SELinux Policies for Non-Standard Configuration

## Purpose

This phase addresses the access restrictions imposed by SELinux when Apache attempts to use resources outside its default policy scope.

Two non-standard changes have been introduced:

- Apache is using port `3131` instead of an approved default HTTP port.
- Apache is serving content from `/var/test_www/html` instead of `/var/www/html`.

Under **SELinux Enforcing mode**, these changes are not automatically permitted. Therefore, the administrator must explicitly update SELinux policy mappings.

---

## Checking the SELinux Mode

Run the following command:

```
getenforce
```

The expected output is:

```
Enforcing
```

This confirms that SELinux is actively enforcing policy decisions rather than simply logging them.

## Allowing Apache to Use Port 3131

Run the following command:

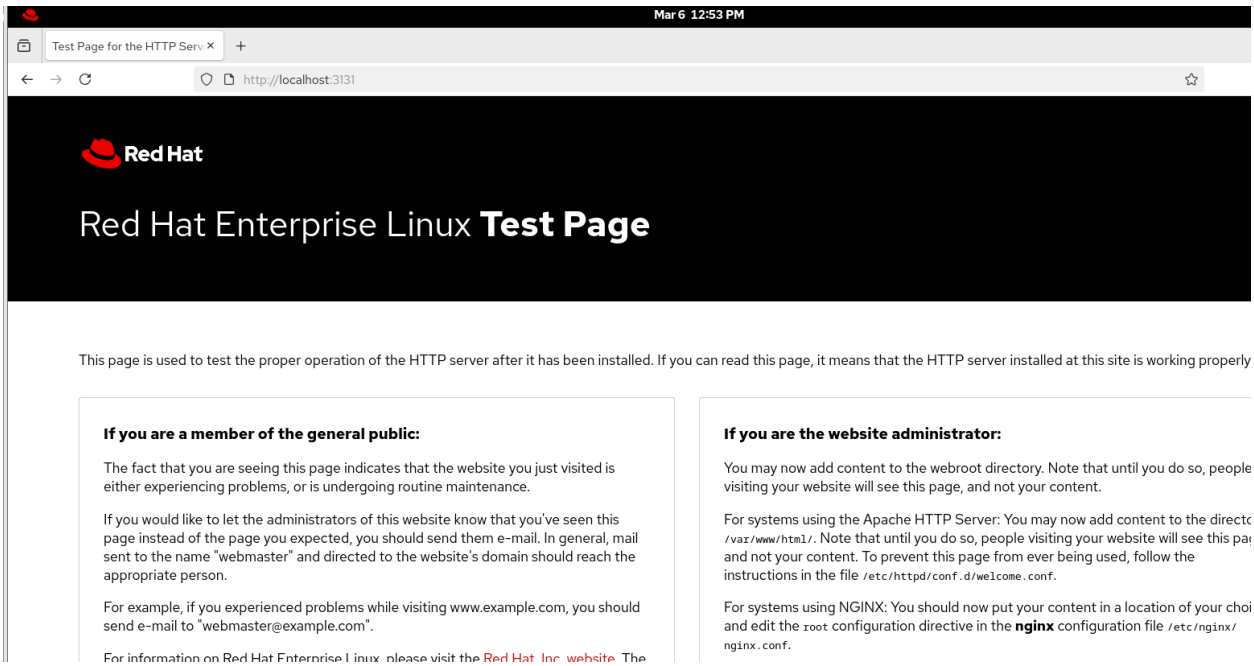
```
sudo semanage port -a -t http_port_t -p tcp 3131
```

### Explanation

- `semanage port` manages SELinux port labeling policy
- `a` adds a new rule
- `t http_port_t` assigns the port to the SELinux type used for HTTP services

- `p tcp` specifies the protocol
- `3131` is the custom port being authorized

## Output




## Security significance

SELinux allows Apache to bind only to ports labeled with the appropriate HTTP port type. Since `3131` is not a default HTTP port, it must be explicitly associated with `http_port_t`.

## Verification

To confirm the change, run:

```
sudo semanage port -l | grep http_port_t

http_port_t                    tcp      3131, 80, 81, 443, 48
8, 8008, 8009, 8443, 9000
http_port_t                    udp      80, 443
pegasus_http_port_t            tcp      5988
```

The expected output should include port `3131`.

## Applying File Contexts to the Custom Web Directory

Run the following commands:

```
sudo semanage fcontext -a -e /var/www /var/test_www

sudo restorecon -Rv /var/test_www
Relabeled /var/test_www from unconfined_u:object_r:var_t:s0 t
o unconfined_u:object_r:httpd_sys_content_t:s0
Relabeled /var/test_www/html from unconfined_u:object_r:var_
t:s0 to unconfined_u:object_r:httpd_sys_content_t:s0
Relabeled /var/test_www/html/index.html from unconfined_u:obj
ect_r:var_t:s0 to unconfined_u:object_r:httpd_sys_content_t:s
0
```

### Explanation of the first command

```
sudo semanage fcontext -a -e /var/www /var/test_www
```

This command tells SELinux to treat `/var/test_www` as equivalent to `/var/www` in terms of file context rules. In effect, it maps the SELinux labeling behavior of the standard Apache content directory to the new custom path.

### Explanation of the second command

```
sudo restorecon -Rv /var/test_www
```

This command applies the correct security labels to the actual files and directories under `/var/test_www` based on the policy mapping.

- `R` applies the operation recursively
- `v` displays each relabeling action

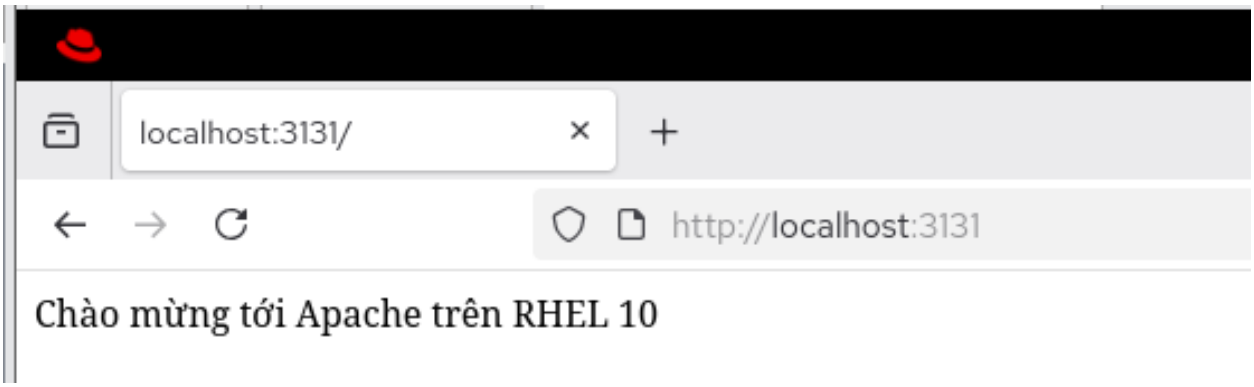
### Verification

To inspect the current SELinux contexts, run:

```
ls -Zd /var/test_www
ls -Z /var/test_www/html
```

If the labels are correct, Apache will be permitted to read the content stored in the custom directory.

### Output




## Security Interpretation

This step is central to the educational value of the lab. In traditional Linux permission management, correct ownership and read permissions might be sufficient to allow a service to access a file. Under SELinux, however, the type of the file and the policy assigned to the process are equally important.

Thus, even if:

- Apache is running
- file ownership is correct
- read permissions are granted

the service may still be denied access if the file is not labeled with a context appropriate for web content.

This demonstrates the practical difference between:

- **Discretionary Access Control (DAC)**, based on ownership and permission bits
- **Mandatory Access Control (MAC)**, based on centrally enforced security policy

SELinux implements the latter and therefore adds an additional layer of protection.

# 8. Phase 4: Firewall Configuration, Persistence, and Verification

## Opening the Firewall for Port 3131

Even if Apache and SELinux are configured correctly, access from other systems will fail if the firewall does not permit inbound traffic on the new port.

Run:

```
sudo firewall-cmd --add-port=3131/tcp --permanent
sudo firewall-cmd --reload
```

### Explanation

- `-add-port=3131/tcp` opens the custom TCP port
- `-permanent` makes the rule persistent across reboots
- `-reload` applies the new configuration

### Verification

Run:

```
sudo firewall-cmd --list-ports
```

The expected output should include `3131/tcp`.

# Enabling Apache at Boot

To ensure that Apache starts automatically whenever the system reboots, run:

```
sudo systemctl enable --now httpd
```

## Explanation

- `enable` configures the service to start automatically at boot
- `-now` starts the service immediately

## Verification

Run:

```
sudo systemctl is-enabled httpd
sudo systemctl status httpd
```

# Local Verification

Test the service locally using:

```
curl http://localhost:3131/index.html
```

The expected output is:

```
Apache custom web root on RHEL 10
```

If this result appears, it confirms that:

- Apache is listening on port `3131`
- the custom web root is functioning

- SELinux no longer blocks access to the content

## Browser-Based Verification

The service can also be tested from a browser by navigating to:

```
http://<server_ip>:3131
```

For example:

```
http://192.168.1.100:3131
```

If the custom page is displayed, the deployment is successful.

# 9. Expected Final Outcome

At the conclusion of the lab, the system should satisfy all of the following:

- Apache is installed and running correctly.
- The service listens on port `3131`.
- Web content is served from `/var/test_www/html`.
- SELinux remains in `Enforcing` mode.
- The firewall allows inbound traffic on `3131/tcp`.
- Access through `curl` and a web browser is successful.

# 10. Conclusion

This lab demonstrates that secure service deployment in an enterprise Linux environment requires more than simply installing software and editing application configuration files. On Red Hat Enterprise Linux, the successful deployment of a service depends on the consistency of several components:

- operating system registration and package availability
- application configuration

- service management
- firewall policy
- SELinux policy and labeling

The most important lesson from this exercise is that SELinux is not an obstacle to deployment, but a structured security mechanism that ensures services operate only within explicitly authorized boundaries. When administrators move a service away from its default configuration, they must extend those boundaries in a controlled and policy-compliant way.

For this reason, the lab serves as both a deployment exercise and a practical introduction to security-aware system administration in enterprise Linux.

# 11. Appendix: Command Summary

```
sudo subscription-manager register --username <your_username>
--password <your_password> --auto-attach

sudo yum install httpd policycoreutils-python-utils setrouble
shoot-server -y

sudo mkdir -p /var/test_www/html
echo "Apache custom web root on RHEL 10" | sudo tee /var/test
_www/html/index.html

sudo vi /etc/httpd/conf/httpd.conf
sudo apachectl configtest

sudo semanage port -a -t http_port_t -p tcp 3131
sudo semanage fcontext -a -e /var/www /var/test_www
sudo restorecon -Rv /var/test_www

sudo firewall-cmd --add-port=3131/tcp --permanent
sudo firewall-cmd --reload
```

```
sudo systemctl enable --now httpd

curl http://localhost:3131/index.html
```